

```
# Assignment 8 – State Value Function and Action Value Function

# Grid World
# S0 S1 G
# X S3 S4

# Rewards
NORMAL_REWARD = -1
GOAL_REWARD = 10
OBSTACLE_REWARD = -5

# Discount factor
gamma = 0.9

# Policy
policy = {
    "S0": "Right",
    "S1": "Down",
    "S3": "Right",
    "S4": "Up"
}

# State transitions
transitions = {
    ("S0", "Right"): "S1",
    ("S1", "Down"): "S3",
    ("S3", "Right"): "S4",
    ("S4", "Up"): "S1"
}

# -----
# Function to get next state and reward
# -----

def get_next_state_reward(state, action):

    # S0 -> Right -> S1
    if state == "S0" and action == "Right":
        return "S1", NORMAL_REWARD

    # S1 -> Down -> S3
    elif state == "S1" and action == "Down":
        return "S3", NORMAL_REWARD

    # S3 -> Right -> S4
```

```

elif state == "S3" and action == "Right":
    return "S4", NORMAL_REWARD

# S4 -> Up -> S1
elif state == "S4" and action == "Up":
    return "S1", NORMAL_REWARD

# S0 -> Up -> Obstacle
elif state == "S0" and action == "Up":
    return "X", OBSTACLE_REWARD

else:
    return None, 0

# -----
# State Value Function
# -----

def state_value_function(start_state, steps=100):

    state = start_state
    total_return = 0
    discount = 1

    for i in range(steps):

        action = policy[state]

        next_state, reward = get_next_state_reward(state, action)

        # Add discounted reward
        total_return += discount * reward

        # If terminal/obstacle/goal is reached
        if next_state == "X" or next_state == "G":
            break

        # Move to next state
        state = next_state

        # Update discount factor
        discount *= gamma

    return total_return

# -----
# Action Value Function

```

```

# ACTION VALUE FUNCTION
# -----

def action_value_function(state, action):

    next_state, reward = get_next_state_reward(state, action)

    # If obstacle or goal is reached
    if next_state == "X" or next_state == "G":
        return reward

    # Calculate future value following the policy
    future_value = state_value_function(next_state)

    return reward + gamma * future_value

# -----
# Calculate State Value for S0
# -----

state_value = state_value_function("S0")

print("State Value Function")
print("-----")
print("V(S0) =", round(state_value, 4))

# -----
# Calculate Action Value for S0 and Up
# -----

action_value = action_value_function("S0", "Up")

print()
print("Action Value Function")
print("-----")
print("Q(S0, Up) =", round(action_value, 4))

```

State Value Function

V(S0) = -9.9997

Action Value Function

Q(S0, Up) = -5

